

OPAUDIT 014 — One crontab per box, one winner per job: why “cron is fixed” never stayed fixed

Audit ID	2026-09-14-014
Report date	2026-09-14
First observed	2026-09-14 morning, on the box that hosts 37 sites: one site had a scheduled fediverse job, thirty-six did not. Afternoon, on the relay box: the relay had no scheduled job of its own either, and its last run was thirteen hours old.
Theme	Every site on a box registers its jobs into the same crontab under the same tag. Registering site B silently replaces site A. The last site registered wins; everyone else runs only when a visitor happens to load a page.
Surfaces	Fediverse delivery, RSS blogroll fetch, version/update check, the photography curator on the relay — every scheduled job, on every multi-site box.
State	RUNNING ON THE BOX — CONFIRMING A box-level scheduler that loops every site directory is installed on both boxes and was watched firing on both. The code that caused this — the shared tag, the “registered” check that reads crontab text, the fleet board that stayed green — is not yet changed. By this ledger’s own rule, cron is not called <i>fixed</i> until a deploy has been watched with every site’s job firing afterwards.
Watched live?	Yes, twice. Sites box: the 15:20 UTC run drained one silent site’s 28 queued deliveries and a reply appeared on pixelfed.social. Relay box: installed 16:26 UTC, the relay’s own Cron & Jobs page went from “13h ago” to “16:50:01, running” on the next tick.
Reporter	Sean (read the crontabs on both boxes, installed both fixes, and said the words “I keep being told this has been definitively fixed”) + Claude (traced the register code to the shared tag; wrote this).
Related	OPAUDIT 011 (jobs registered but pointing at old paths) — this audit corrects 011 : the re-registration described there as done on all 24 spokes could never have held, for the reason in §3. OPAUDIT 003 (background jobs must not run inside web requests) — the page-hit fallback is exactly that, and it is what kept the fleet looking alive.
Publication	Public. No exploitable detail.

1. SUMMARY

SnapSmack sites register their own scheduled jobs. Each site writes a line into the web server user’s crontab, marked with a tag so the site can find it again. The tag is the same for every site: `# snapsmack-fediverse`. On a box with one site, that works. On a box with thirty-seven sites sharing one web user, there is one crontab and one tag — so there is room for exactly one line. Whichever site registered last owns it. The other thirty-six see “a tagged line exists but the command isn’t mine,” report that, and never install their own. They were never scheduled. They ran only when somebody loaded a page.

2. WHAT A HUMAN SAW

Cron “fixed again.” Posts federated while Sean was looking at the site and stopped when he wasn’t. A fleet cron board that showed green everywhere. A site with 28 deliveries sitting in its queue for hours. On the relay, the curator frozen at 03:08 and the relay’s own delivery job thirteen hours stale, while its Cron & Jobs page said the entry was “registered but invalid” and that a fleet tick would repair it automatically. It had said that for thirteen hours.

3. ROOT CAUSE

Three things, one underneath the other.

The shared tag. The register routine finds “its” job by looking for the tag in `crontab -l` for the current user. Every site on the box runs as the same user and uses the same tag. So registration is not per-site; it is per-box, and the box can hold one site’s job. This is why OPAUDIT 011’s fleet-wide re-registration through each site’s Cron & Jobs page undid itself as it went: each spoke’s REGISTER replaced the previous spoke’s line. The last one through the door — `wateronthebrain.ca` — was the only site with a job this morning.

“Registered” reads text, not evidence. The status the page shows comes from parsing crontab text. It cannot see jobs scheduled any other way (the fix below lives in `/etc/cron.d`, invisible to `crontab -l`), and it says nothing about whether the job has actually fired. OPAUDIT 011 added a command-level check; that was an improvement to the wrong instrument.

The fleet board could not go red. The fleet status API reports a `last_run` timestamp per job. The page-hit fallback — the web request that drains the queue when no cron has — writes the *same* timestamp. So every visitor kept the number fresh, and CRONOMETER showed thirty-six unscheduled sites as healthy. There is a separate timestamp written only by the real command-line run; nothing displays it.

4. EVIDENCE AT DIAGNOSIS

Sites box, 37 site directories: one self-registered `cron-fediverse.php` line (`wateronthebrain.ca`); three stale lines still calling a script by its pre-rename name; version-check lines for 13 sites; nothing at all for the rest. Relay box: the web user’s crontab held three tagged lines, all for `photofri.day`; nothing for `photoblogs.fyi` itself; the relay’s page reported “tagged entry exists, but it is invalid: registered command does not match this job” for all three jobs, last runs 02:54 / 02:00 / 02:57 UTC, read at 16:24 UTC.

5. WHAT WE DID

On both boxes: installed `/etc/cron.d/snapsmack-fleet` — three lines owned by root, one per job, each looping every site directory under the web root, skipping directories without the script, and running each site’s job under a per-site lock so runs never overlap. This does not depend on any site registering anything, survives deploys, and adds a new site the moment its directory exists. The old per-site crontab lines were left in place for now; the delivery sweep’s own database lock stops a double drain, and they are on the cleanup list.

In the code (0.7.712D, built the same afternoon, not yet deployed): (a) crontab lines are matched by tag *and* script path, so a site's REGISTER, UNREGISTER, the post-deploy refresh and the fleet tick's auto-repair touch only that site's line; an orphan line whose script is gone is adopted, another site's is never touched; the two pages that still edited the crontab by hand now go through the one helper. (b) Each cron script stamps a scheduler heartbeat first thing, and only when the scheduler launched it — RUN NOW and event kicks are marked manual and ignored, the page-hit fallback is not command-line and ignored. Cron & Jobs gets a SCHEDULER row that reads that stamp alone: FIRING, or NOT FIRING / NEVER FIRED / HAS NOT FIRED SINCE THE DEPLOY, in red, in words. (c) The fleet heartbeat ships the same verdict per job and CRONOMETER judges on it when present. (d) Every finished deploy is stamped; a job that has not fired within fifteen minutes of it is red on every surface. (e) The cleanup paste is still owed.

6. VERIFIED LIVE — AND WHAT REMAINS

Sites box: installed 14:55 UTC; the 15:20 run drained allinthewrist.photoblogs.fyi's 28 queued deliveries and the reply they carried appeared on pixelfed.social. Relay box: installed 16:26:55 UTC; on the next ten-minute tick the relay's own page went from "13h ago" to "3m ago (16:50:01, running)". Hourly and six-hourly jobs on the relay were not yet due at the time of writing.

Relay, later the same afternoon: the hourly RSS job fired at 17:07:01 on the new schedule and delivery kept firing on the tens — two jobs, watched. What remains is the deploy. Until 712D ships and a deploy is watched with every site's SCHEDULER row green afterwards, this stays **RUNNING ON THE BOX — CONFIRMING**. The status pages will keep saying "registered but invalid" until then; that message is now known to be the instrument, not the patient.

7. EYEBALL NEXT

Relay: the 17:07 RSS tick and the 18:23 version tick. Sites box: the `cron-smackverse.php` lines and duplicate per-site lines removed in one paste. Then the build: after it deploys, every site's Cron & Jobs page shows a heartbeat in minutes, CRONOMETER shows the same, and a deliberately un-scheduled test site shows red within fifteen minutes.

8. THE DOCTRINE THIS INCIDENT CREATED

A job is running when its heartbeat moves, and for no other reason. Not because a line exists, not because the line's command is right, not because a status page says so, and not because the queue is empty — a visitor may have emptied it. Every scheduled job writes its own heartbeat at the start of every run; every status surface reads that heartbeat and nothing else. And anything a site registers for itself must carry the site's name, or on a shared box it is registering for whoever comes last.